Text.Regex.TDFA: https://hackage.haskell.org/package/regex-tdfa

Regex formats

POSIX extended regex

```
"abc"
"[a-z]"
".*"
"a+"
"a*"
"a?"                  0 or 1 match
"a{2,4}"              2~4 match
"a|b"                 OR
"[[:lower:]]"         class
"(a*)"                Capture

Escapes
"\""                  Quote
"\*"                  ERROR
"\\*"                 OK
"\\+"
"\\."
"\\(abc\\)"
"\\[abc\\]"
```

Basics

```
import Text.Regex.TDFA

"abcde" =~ "cd" :: Bool
 -- True
"abcde" =~ "cd" :: Int
 -- 1. Number of matches
"abbbe" =~ "b" :: Int
 -- 3
"abbbe" =~ "bb" :: (MatchOffset, MatchLength)
 -- (1, 2) i.e. index=1 length=2
"abcde" =~ "cd" :: String
 -- "cd"
"abcde" =~ "z" :: String
 -- "". No match

"abcde" =~ "cd" :: (String, String, String)
 -- ("ab", "cd", "e")    before, match, after
"abbbbe" =~ "b" :: (String, String, String)
 -- ("ab", "b", "bbbe")  first match only if multiple
"abbbbe" =~ "z" :: (String, String, String)
 -- ("abbbe", "", "")    no match

getAllTextMatches ("abbbbe" =~ "b") :: [String]
              -- ["b", "b", "b", "b"]   all matches
getAllMatches ("abbbbe" =~ "b") :: [(MatchOffset,
MatchLength)]    -- [(1,1),(2,1),(3,1),(4,1)]
```

```
"abcbde" =~ "b(.)" :: [[String]]
 -- [["bc", "c"],["bd","d"]]
getAllTextSubmatches ("abcbde" =~ "b(.)") :: [String]
  -- ["bc", "c"] First only
getAllSubmatches ("abcbde" =~ "b(.)") :: [(MatchOffset,
MatchLength)]    -- [(1,2),(2,1)]


-- others
"abcde" =~ "cd" :: MatchArray
"abcde" =~ "cd" :: [MatchArray]
"abcde" =~ "cd" :: [MatchText String]
```